RAPPORT D'INGÉNIERIE DES DONNÉES · PROJET 3

Faut-il vendre au Brésil ?

Une modern data stack locale sur 100 000 commandes réelles : data lake S3, dbt, DuckDB, Dagster, Docker et intégration continue

LUMU BAMANA MANASSE

Data engineer

Date	Octobre 2026
Version	1.0.0 (livrée et recettée)
Client	Maison Caju (entreprise fictive) · Mission CAJ-2026-001
Dépôt GitHub	https://github.com/MediJr/olist-ecommerce-modern-data-stack
Langue	Français · *English version available in the repository*

Scénario fictif, données réelles : l'entreprise cliente et ses interlocuteurs sont inventés pour simuler une mission professionnelle complète. Les données sont celles du *Brazilian E-Commerce Public Dataset by Olist*, publié sur Kaggle sous licence CC BY-NC-SA 4.0 (commandes anonymisées de 2016 à 2018). Ce projet n'est ni commandé, ni validé, ni affilié à Olist.

Fiche du document

Projet	Projet 3 du portfolio : modern data stack (data lake, dbt, orchestration, CI)
Mission	CAJ-2026-001 : « Faut-il se lancer sur les marketplaces brésiliennes ? »
Client	Maison Caju, marque française de linge de maison (fictive), à l'attention de Claire Dumont
Auteur	Lumu Bamana Manasse, data engineer
Période de réalisation	Octobre 2026
Données	Olist, 99 441 commandes passées de septembre 2016 à octobre 2018
Statut	LIVRÉ RECETTÉ
Code source	https://github.com/MediJr/olist-ecommerce-modern-data-stack

Historique des versions

Version	Date	Objet
0.1	23/10/2026	Cadrage : brief, compte rendu, cahier des charges
0.5	23/10/2026	Lac S3, ingestion, projet dbt, orchestration Dagster
0.9	25/10/2026	Passage de toute la plateforme dans Docker, intégration continue, tableau de bord
1.0.0	25/10/2026	Recette sur poste neuf, rapport final (français et anglais)

Guide de lecture

Comité de direction	1. Résumé exécutif et 11. Résultats et recommandations
Analyste (Hugo)	6. Modèle de données dbt, 7. Qualité et tests, 9. Exploitation
Ingénieur reprenant le projet	4. Architecture, 5. Pipeline et orchestration, 13. Retour d'expérience, annexes

Table des matières

Fiche du document 2

1 Résumé exécutif 4

2 Contexte et cadrage 5

2.1 Ce que la demande implique techniquement 5

2.2 Décisions du cadrage 5

2.3 Questions business 5

3 Sources et profilage 6
3.1 Le jeu de données 6
3.2 Ce que le profilage a révélé 6

4 Architecture 7

5 Pipeline et orchestration 7
5.1 Le contrat de livraison 7
5.2 La couche bronze : idempotente par construction 8
5.3 L'orchestration par événements 8

6 Modèle de données dbt 8
6.1 Trois couches, trois rôles 8
6.2 Le schéma en étoile 9
6.3 La règle qui évite de multiplier le chiffre d'affaires 9

7 Qualité et tests 9
7.1 Une intégration continue qui mord 10

8 Sécurité et données personnelles 10

9 Exploitation 11

10 Recette 11

11 Résultats et recommandations 12

12 Limites 15

13 Retour d'expérience 16

14 Feuille de route 16

A Arborescence du dépôt 17

B Étapes de l'intégration continue 17

1 Résumé exécutif

Le besoin. Maison Caju envisage de vendre ses collections au Brésil via les marketplaces locales. Son comité de direction veut une étude de marché chiffrée, et son analyste doit pouvoir **faire évoluer les indicateurs seul, en SQL, sans risque de régression**. Les données arrivent par **livraisons mensuelles**.

La solution. Une **modern data stack** complète qui tient sur un poste et démarre avec une seule commande, `docker compose up` : chaque livraison est contrôlée puis rangée dans un **data lake S3** (Parquet, une partition par mois). Elle est ensuite transformée par **dbt** sur **DuckDB** en un schéma en étoile testé, orchestrée par **Dagster** (capteurs, partitions, rejeux), et servie à un tableau de bord bilingue. Une **intégration continue** reconstruit et teste toute la chaîne à chaque modification.

99 441 commandes réelles (2016-2018)	**27 / 27** exécutions réussies sur poste neuf	**18 min** de zéro au tableau de bord	**94 / 94** modèles et tests dbt
0 doublon au rejeu	**= source** CA rapproché au centime	**14** tests Python	**6** décisions documentées (ADR)

Principaux résultats pour Maison Caju

— **Q1 · Où vendre :** 13,5 M BRL de chiffre d'affaires. L'univers maison pèse **19,8 %**, et le linge de maison (*bed_bath_table*) est la 3e catégorie. São Paulo, Rio et Minas Gerais font **63 %** du CA.

— **Q2 · Fidélité :** seuls **3,0 %** des clients commandent deux fois. Ce marché se gagne par l'acquisition.

— **Q3 · Livraison :** la note passe de **4,3 à 2,1** dès 4 jours de retard. Le choix du transporteur est stratégique.

— **Q4 · Paiement :** carte bancaire à 75,5 %, *boleto* à 19,9 %. Le panier triple avec le paiement en 7 à 10 fois.

— **Q5 · Fret :** 13,8 % du prix à São Paulo, jusqu'à **28,5 %** dans le Nord.

Ce que démontre ce projet, côté ingénierie : data lake S3 et format Parquet partitionné, contrat de livraison, ingestion idempotente, modélisation dbt en couches avec tests de données, tests unitaires, contrat et rapprochements avec la source, orchestration par événements avec Dagster, gestion de la concurrence d'un entrepôt à écrivain unique, conteneurisation complète et intégration continue.

2 Contexte et cadrage

2.1 Ce que la demande implique techniquement

Demande du client	Conséquence technique
« Hugo doit faire évoluer les indicateurs sans casser ce qui marche »	Logique métier en **SQL versionné et testé** (dbt), **intégration continue** obligatoire
« Un jeu de données très détaillé » (9 tables)	**Modèle dimensionnel** : grain explicite, schéma en étoile
« Un tableau de bord pour le comité »	Il lit des tables **déjà calculées et testées**, jamais les fichiers bruts
Exports mensuels du partenaire	**Partitions** par livraison, **rejeu** sans doublon, **orchestrateur**

Table 1 – De la demande aux exigences techniques

2.2 Décisions du cadrage

#	Sujet	Décision
D1	Mises à jour	Un export par mois ; simulé en découpant le jeu public par mois d'achat
D2	Chiffre d'affaires	Prix des articles, hors fret, commandes ni annulées ni indisponibles. Défini **une seule fois**, dans dbt
D3	Client fidèle	Client identifié par `customer_unique_id` ; fidèle = au moins deux commandes valides
D4	Tableau de bord	Metabase, remplacé ensuite par Streamlit (ADR 004)
D5	Maintenance	Toute modification passe par une *pull request* testée par l'intégration continue
D6	Langue	Tables et colonnes en anglais ; documentation en français, rapport aussi en anglais

Table 2 – Décisions de la réunion de cadrage du 23/10/2026

2.3 Questions business

#	Question	Décision éclairée
Q1	Quelles catégories et quels États font le CA, et comment évolue-t-il ?	Collections et régions de lancement
Q2	Quelle part des clients revient acheter ?	Fidélisation ou acquisition
Q3	Le retard de livraison dégrade-t-il la note ?	Choix du transporteur
Q4	Comment les clients paient-ils ?	Offre de paiement
Q5	Combien pèse le fret selon l'État ?	Coût logistique par région

Table 3 – Les cinq questions retenues

3 Sources et profilage

3.1 Le jeu de données

Le *Brazilian E-Commerce Public Dataset by Olist* décrit environ 100 000 commandes anonymisées passées sur une marketplace brésilienne entre 2016 et 2018. Il compte **9 fichiers CSV liés** : commandes, articles, paiements, avis, clients, vendeurs, produits, géolocalisation et traduction des catégories. Le client l'a téléchargé sur Kaggle (compte personnel requis), puis déposé dans `data/source/`.

3.2 Ce que le profilage a révélé

Constat	Risque si on l'ignore, et traitement
`customer_id` : 99 441 valeurs pour 96 096 personnes (`customer_unique_id`)	Il change à chaque commande : la fidélité paraîtrait **nulle**. Compter sur `customer_unique_id`
2 961 commandes réglées en plusieurs paiements (jusqu'à 29) ; 9 803 commandes à plusieurs articles	Une jointure articles × paiements **multiplie le CA**. Agréger chaque table avant de joindre
98 410 `review_id` pour 99 224 avis	Le `review_id` seul n'est pas une clé : clé composite (`review_id`, `order_id`)
2 catégories sans traduction, 610 produits sans catégorie	Cases vides dans le tableau de bord : repli explicite et testé (`unknown`)
775 commandes sans article ; 8 commandes « livrées » sans date de livraison	Exclues du CA ou des délais, de façon explicite
1 000 163 points GPS pour 19 015 préfixes de code postal, dont certains hors du Brésil	Médiane par préfixe, après filtrage géographique
Novembre 2016 : aucune commande ; septembre-octobre 2018 : 20 commandes, presque toutes annulées	Une livraison vide reste une livraison ; analyses sur les mois complets (01/2017-08/2018)

TABLE 4 – Profilage : sept pièges, sept traitements

4 Architecture

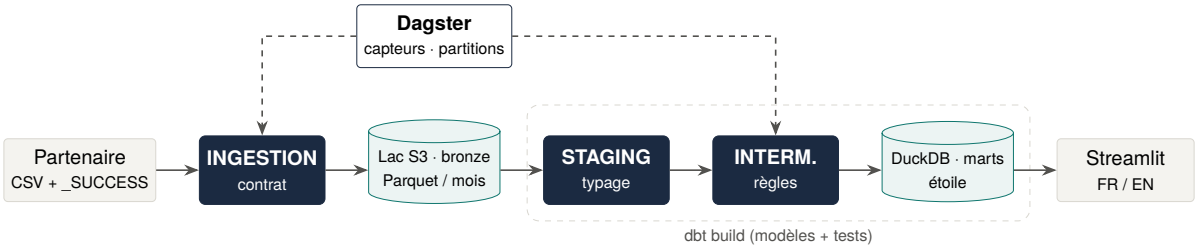


FIGURE 1 – De la livraison au tableau de bord

Rôle	Choix	Pourquoi
Data lake	Stockage compatible S3 (SeaweedFS), Parquet	Même interface qu'Amazon S3 ou Google Cloud Storage : seul l'emplacement change pour passer au cloud
Entrepôt	DuckDB	Colonnaire, lit le Parquet sur S3 directement, rien à administrer
Transformations	dbt 1.12	SQL versionné, testé, documenté, avec le lignage : maintenable par un analyste
Orchestration	Dagster 1.13	Raisonne en *assets* ; chaque modèle dbt devient un asset et chaque test un contrôle
Restitution	Streamlit	Tableau de bord versionné et testé, lecture seule sur l'entrepôt
Exécution	Docker Compose	Trois services ; image unique `python:3.13-slim`, versions figées
Qualité	GitHub Actions	Toute la chaîne reconstruite et testée à chaque *pull request*

TABLE 5 – Choix techniques (ADR 001, 002, 004, 005)

Le lac est désigné par une seule variable, `LAKE_URI` : `s3://olist-lake` en exploitation, un simple dossier dans l'intégration continue. **Le code ne fait pas la différence** : c'est ce qui le rend transposable sur le cloud.

5 Pipeline et orchestration

5.1 Le contrat de livraison

Le partenaire dépose un dossier par mois. Il écrit le fichier `_SUCCESS` **en dernier** : un dossier sans ce marqueur est une livraison en cours, jamais lue. Avant toute écriture dans le lac, chaque fichier doit respecter son contrat :

— un **en-tête identique** à celui attendu : une colonne ajoutée, renommée ou déplacée bloque l'ingestion (dérive de schéma) ;

— des **clés non vides et uniques** ;

— une lecture **tout en texte**, avec des colonnes déclarées : un mois vide (en-tête seul) se lit comme un mois plein.

5.2 La couche bronze : idempotente par construction

Chaque table d'un mois devient **un seul fichier Parquet**, rangé selon la convention Hive, que DuckDB, Spark et BigQuery lisent tous :

```
s3://olist-lake/bronze/orders/delivery_month=2017-01/orders.parquet
```

Rejouer un mois **remplace** ce fichier : un doublon est impossible. Deux colonnes de traçabilité sont ajoutées : `_source_file` (fichier d'origine) et `_ingested_at` (heure d'entrée dans le lac).

5.3 L'orchestration par événements

Mécanisme Dagster	Rôle
Partitions dynamiques	Une partition `delivery_month` est créée à la **réception** d'une livraison, plutôt qu'un calendrier fixé à l'avance
Capteur `landing_sensor`	Toutes les 30 s : une livraison nouvelle **ou relivrée** (marqueur modifié) déclenche le job `monthly_delivery`
Clé d'exécution `mois@horodatage`	Une même version de livraison n'est jamais traitée deux fois
Limite de concurrence	DuckDB n'accepte qu'un écrivain : **une exécution à la fois** (`dagster.yaml`)
Amorçage	Le capteur mensuel attend les référentiels ; des référentiels arrivés seuls sont chargés sans dbt
Délai maximal	15 minutes par exécution, surveillées par le démon : un blocage ne retient plus la file

Table 6 – Orchestration (ADR 006)

6 Modèle de données dbt

6.1 Trois couches, trois rôles

Couche	Modèles	Rôle
Staging (vues)	9	Un modèle par source : renommer, typer (`decimal` pour l'argent, jamais `float`), garder la dernière livraison. **Aucune règle métier**
Intermediate (vues)	2	`int_orders` : une ligne par commande et **toutes** les règles métier, écrites une seule fois
Marts (tables)	6 + 5	Schéma en étoile, plus une table de reporting par question

Table 7 – Les 22 modèles dbt (ADR 003)

6.2 Le schéma en étoile

La table de faits principale, `fct_order_items`, a pour **grain l'article vendu** : c'est le niveau le plus fin qui porte le chiffre d'affaires. On peut toujours agréger vers le haut (commande, catégorie, mois), jamais redescendre. Elle est entourée de `dim_customers` (client réel), `dim_products`, `dim_sellers` et `dim_date`. Une seconde table de faits, `fct_orders`, porte les mesures de la commande : livraison, avis, paiement.

6.3 La règle qui évite de multiplier le chiffre d'affaires

Une commande à 2 articles et 2 paiements produit **4 lignes** si l'on joint les tables brutes. `int_orders` agrège donc chaque table enfant **avant** la jointure :

```
with items as (
    select order_id, count(*) as item_count, sum(price) as items_value,
           sum(freight_value) as freight_value
    from {{ ref('stg_order_items') }} group by order_id
),
payments as (
    select order_id, sum(payment_value) as payment_value, count(*) as payment_count
    from {{ ref('stg_order_payments') }} group by order_id
)
select ... from {{ ref('stg_orders') }} o
left join items using (order_id) left join payments using (order_id)
```

Cette règle est verrouillée par un **test unitaire dbt** (deux articles, deux paiements, montants attendus) et par un **rapprochement** avec la source (section 7).

7 Qualité et tests

Type	Ce qui est vérifié	Nombre
Tests de données dbt	Unicité, non-nullité, relations, valeurs acceptées (statuts, notes, 27 États)	69
Tests unitaires dbt	Validité et retard ; non-multiplication des montants ; rang du client réel	3
Rapprochements (tests singuliers)	CA des marts = total bronze ; paiements = total bronze ; cohorte complète au mois 0	3
Contrat imposé	Colonnes et types de `fct_order_items`	1
Tests Python (pytest)	Découpage des livraisons, contrat, idempotence, mois vide, tableau de bord	14

TABLE 8 – Les filets de sécurité

Le rapprochement avec la source

Le test `assert_revenue_reconciles_with_source` recalcule le chiffre d'affaires **directement sur la couche bronze**, sans passer par aucun modèle dbt, et le compare à celui du tableau de bord. Résultat : 13 494 400,74 **BRL des deux côtés**, au centime près. Si une jointure perd ou duplique un article, les deux totaux divergent et la construction échoue.

7.1 Une intégration continue qui mord

Pour la recette, une erreur classique a été introduite volontairement : **ajouter le fret au chiffre d'affaires**.

dbt build · erreur volontaire

```
# 1. fret ajouté : le type de la colonne change
Compilation Error in model fct_order_items
  This model has an enforced contract that failed.

# 2. fret ajouté, en gardant le même type
94 of 94 FAIL 1 assert_revenue_reconciles_with_source ......... [FAIL 1]
Done. PASS=93 WARN=0 ERROR=1 SKIP=0 TOTAL=94          (code de sortie : 1)
```

Les deux filets se complètent : le **contrat** protège la forme (colonnes, types), le **rapprochement** protège le fond (les montants). Dans les deux cas, dbt sort en erreur : la *pull request* serait refusée.

8 Sécurité et données personnelles

— Données **anonymisées par l'éditeur** : identifiants hachés, ni nom ni adresse ; seuls la ville, l'État et le préfixe du code postal subsistent.

— Secrets (clés S3) dans `.env`, exclu de Git ; le fichier d'identités S3 est généré au démarrage du conteneur.

— Services exposés uniquement sur `127.0.0.1` ; conteneurs exécutés avec un **utilisateur sans privilèges**.

— Le tableau de bord monte l'entrepôt **en lecture seule**.

— Licence CC BY-NC-SA 4.0 : l'archive complète n'est pas redistribuée ; seul un échantillon de 450 commandes, attribué, sert à la CI.

9 Exploitation

Besoin	Commande ou outil
Démarrer la plateforme	`docker compose up -d --build`
Recevoir une livraison	Déposer le dossier du mois dans `data/landing/` : le capteur fait le reste
Rejouer un mois corrigé	Relivrer le dossier (nouveau `_SUCCESS`) ou relancer la partition dans Dagster
Suivre les exécutions	Interface Dagster, http://127.0.0.1:3000 : graphe, partitions, contrôles, journaux
Modifier un indicateur	Modifier le modèle dbt, puis `docker compose run --rm dagster python -m olist_pipeline.dbt_cli build`
Lancer les tests Python	`docker compose run --rm dagster pytest`

TABLE 9 – Gestes d'exploitation

10 Recette

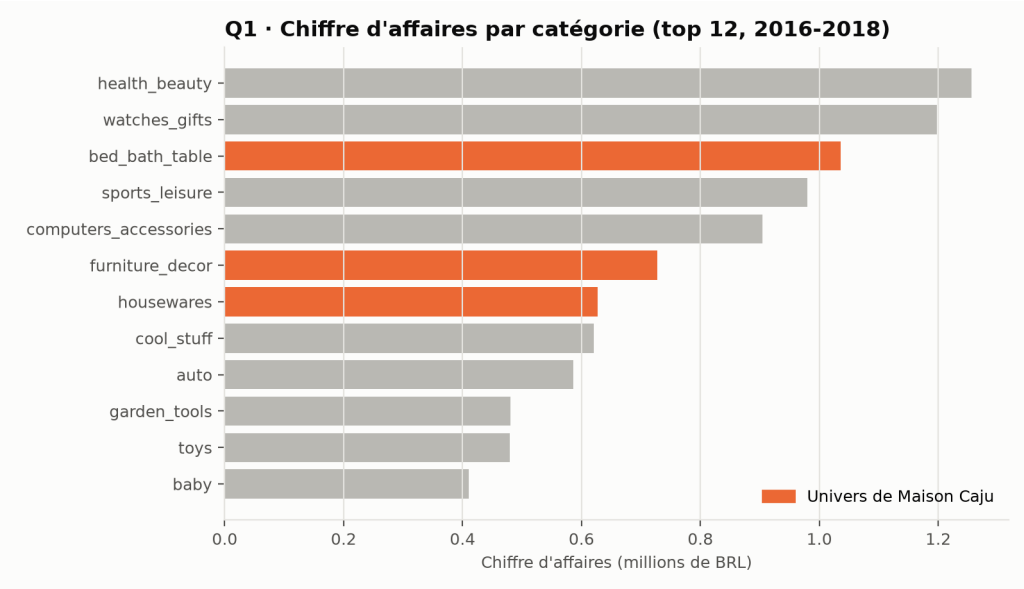
La recette a été passée sur un **poste neuf simulé** : volumes Docker supprimés, zone de dépôt vide. Les 26 livraisons et les référentiels ont été déposés d'un coup, puis traités sans intervention : **27 exécutions réussies sur 27, en 18 minutes**, avec 19 s de traitement médian par mois.

#	Critère	Résultat
C1	Lancement sur un poste neuf en suivant le README	✓
C2	Un mois = une partition ; traiter un mois ne retraite pas les autres	✓
C3	Rejouer un mois ne crée aucun doublon (octobre 2016 relivré : 329 commandes avant et après)	✓
C4	CA des marts égal au total de la source (13 494 400,74 BRL)	✓
C5	Tous les tests dbt passent (94/94) ; chaque modèle est documenté	✓
C6	L'intégration continue échoue si un test casse (démonstration ci-dessus)	✓
C7	Le tableau de bord répond à Q1-Q5 (5 onglets, 10 graphiques, FR/EN)	✓
C8	Aucun secret ni donnée brute versionné	✓

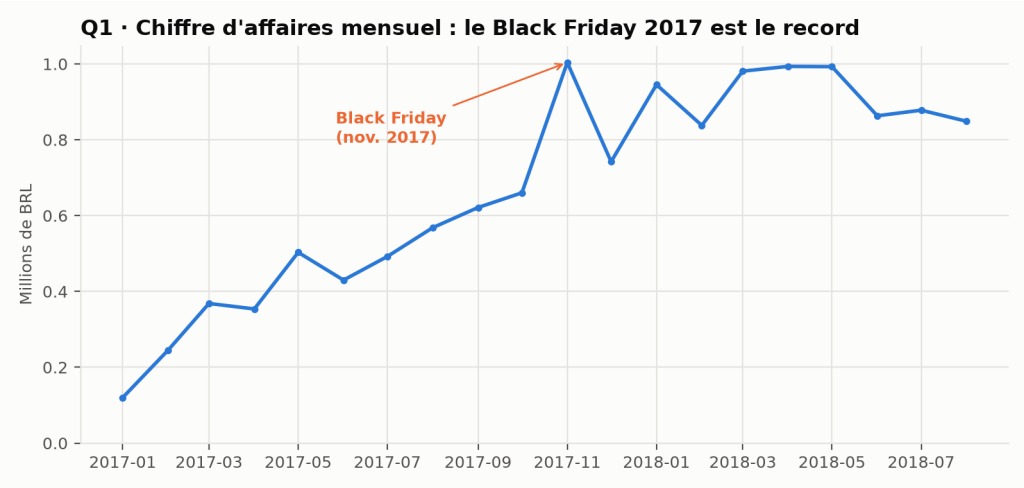
TABLE 10 – Critères d'acceptation : 8 sur 8

11 Résultats et recommandations

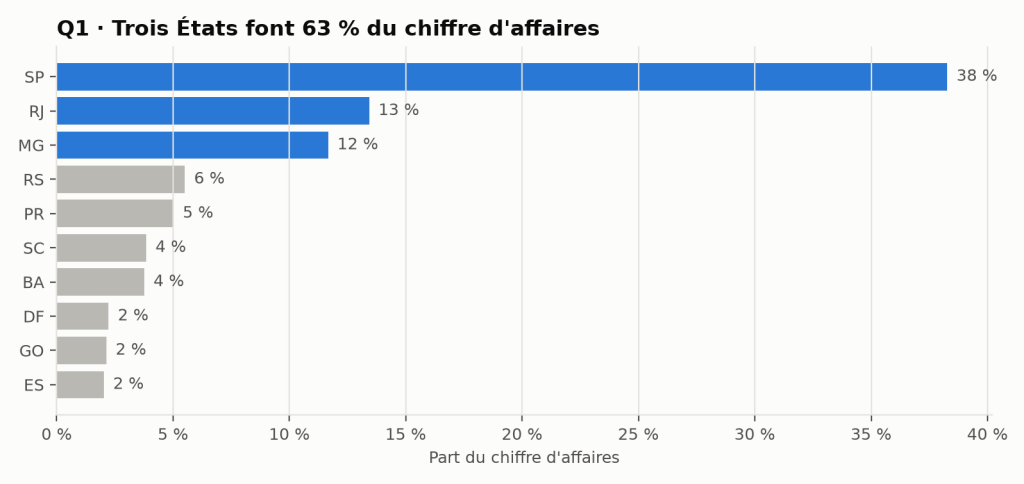
Q1 · Où se fait le chiffre d'affaires ?



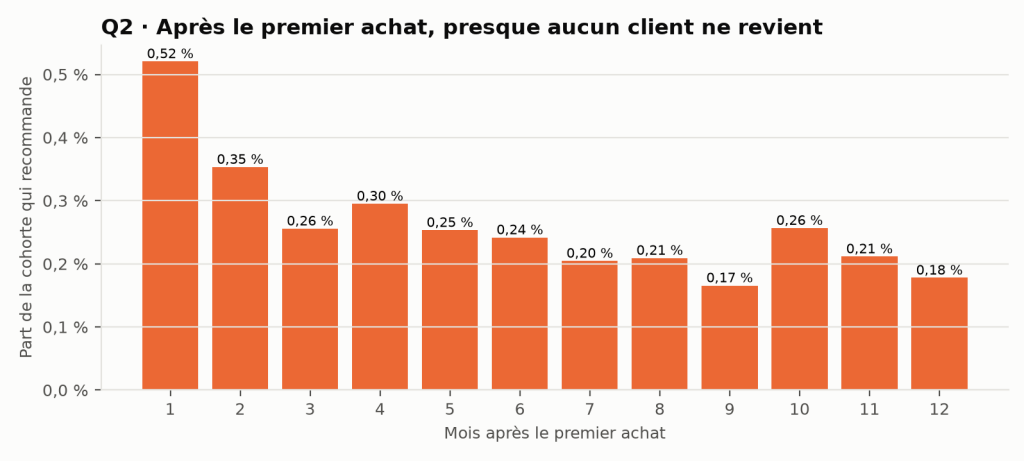
Sur 98 207 commandes valides, le chiffre d'affaires atteint **13,5 M BRL** (hors fret). L'univers de Maison Caju pèse **19,8 %**, et le linge de maison (*bed_bath_table*, 7,7 %) est la 3e catégorie du marché.



Le marché croît fortement en 2017, puis se stabilise autour de 0,9 M BRL par mois en 2018. Le **Black Friday 2017** dépasse octobre de **52 %**.

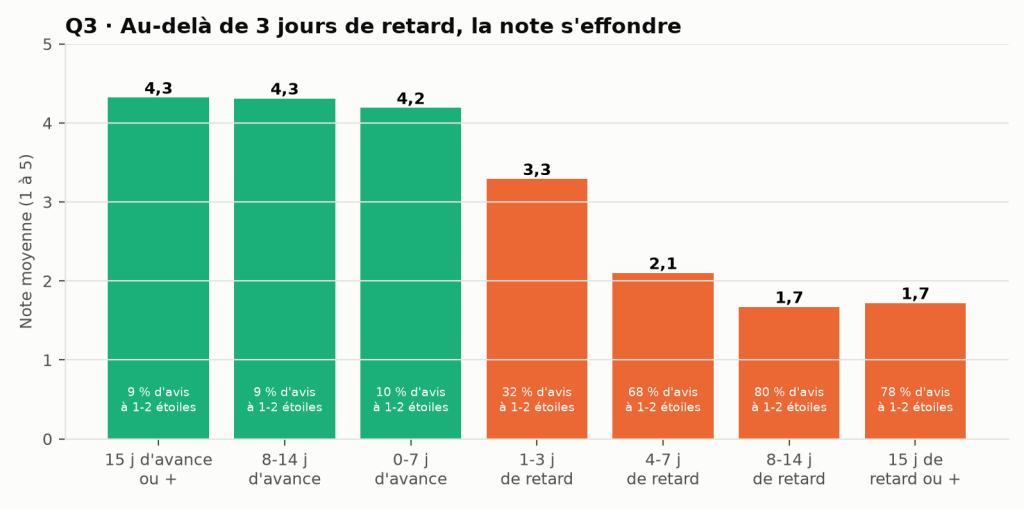


Q2 · Les clients reviennent-ils ?



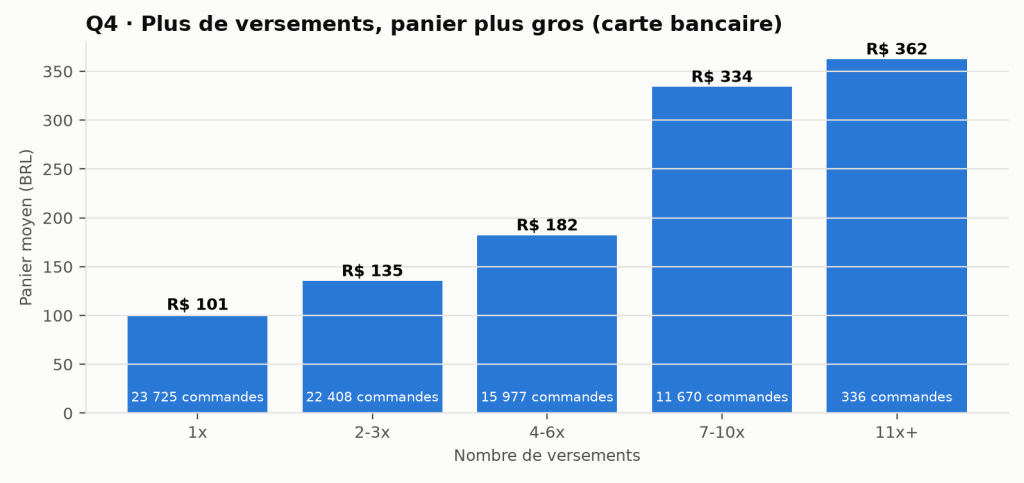
Sur 94 990 clients ayant passé une commande valide, seuls 2 888 (**3,0 %**) en ont passé une deuxième, 28 jours plus tard en médiane. Un mois après leur premier achat, à peine **0,5 %** des clients de 2017 recommandent.

Q3 · Le retard de livraison coûte-t-il des avis ?



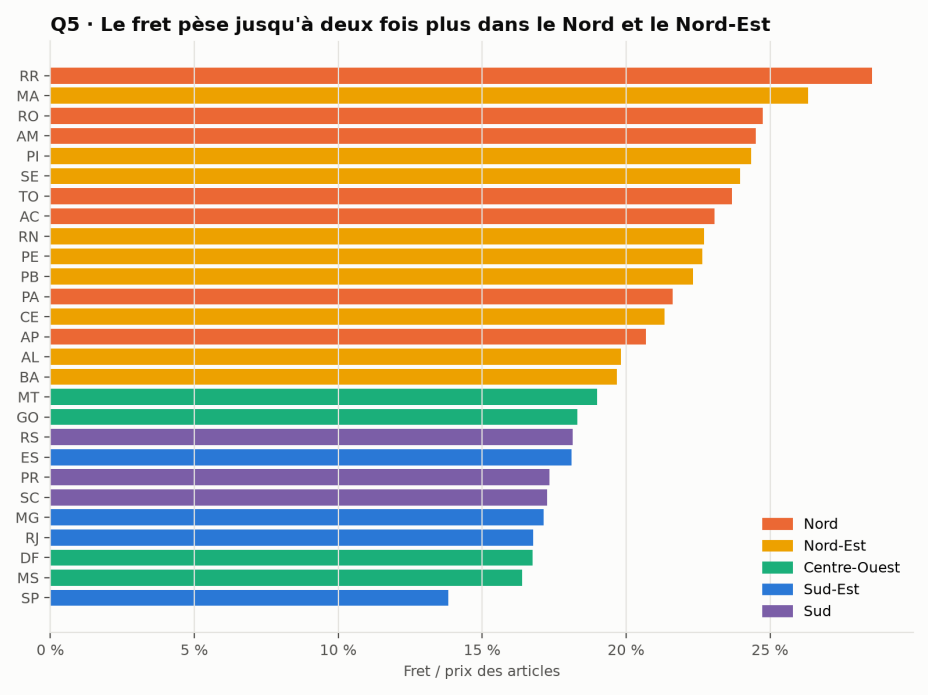
6,8 % des commandes livrées arrivent en retard. La note moyenne tombe de **4,3** (à l'heure ou en avance) à **2,1** entre 4 et 7 jours de retard. Au-delà d'une semaine, **80 %** des avis sont à 1 ou 2 étoiles.

Q4 · Comment les clients paient-ils ?



La carte bancaire règle 75,5 % des commandes, le *boleto* (bordereau payé en espèces ou en ligne) 19,9 %. Avec la carte, le panier moyen passe de **101 BRL** en un versement à **334 BRL** en 7 à 10 versements.

Q5 · Combien pèse le fret ?



Le fret représente 13,8 % du prix des articles à São Paulo (8,7 jours de livraison en moyenne), mais **jusqu'à 28,5 %** dans le Roraima (29 jours). Le Nord et le Nord-Est cumulent fret élevé, délais longs et retards plus fréquents.

Recommandations au comité de direction

1. **Lancer avec le linge de maison et la décoration**, déjà 3e et 6e catégories du marché.
2. **Commencer par le Sud-Est** (São Paulo, Rio, Minas Gerais) : 63 % du CA, le fret le plus faible et les délais les plus courts. Le Nord et le Nord-Est viendront ensuite, avec une politique de fret dédiée.
3. **Investir dans l'acquisition plutôt que dans la fidélisation** : 97 % des clients n'achètent qu'une fois. Préparer le Black Friday.
4. **Choisir le transporteur sur sa ponctualité** et promettre des délais réalistes : un retard de quelques jours divise la note par deux.
5. **Proposer le paiement en plusieurs fois et le *boleto*** : le panier moyen triple avec les versements.

12 Limites

— Les données couvrent 2016-2018 : elles décrivent la **structure** du marché, pas son état actuel. Elles proviennent d'une seule marketplace.

— Le chiffre d'affaires est celui de tous les vendeurs de la marketplace, pas celui d'une marque comparable à Maison Caju.

— L'entrepôt est reconstruit à chaque livraison (environ 20 s) : adapté à ce volume, pas à des centaines de millions de lignes.

— `dagster dev` regroupe serveur et démon dans un conteneur, avec un historique en SQLite : adapté à un poste, pas à la production.

13 Retour d'expérience

Incident	Leçon
L'image MinIO n'est plus publiée	Un composant open source peut changer de modèle de distribution. Grâce au standard S3, SeaweedFS l'a remplacé **sans changer une ligne de code** (ADR 002)
dbt lancé avant la première commande	L'ordre de démarrage d'une plateforme est une dépendance comme une autre : il doit être **explicite** (amorçage, ADR 006)
`dagster-dbt` exige Python < 3.14	Vérifier la compatibilité de **toute** la chaîne avant de choisir une version
Smart App Control bloque des DLL non signées ; une exécution figée 83 min bloque la file	Une file à exécution unique a besoin d'un **délai maximal**. La conteneurisation rend la plateforme indépendante du poste (ADR 005)
Le test du tableau de bord échoue dans la CI	La période par défaut était écrite en dur : un test sur d'autres données révèle les **hypothèses cachées**
Simulation des livraisons en 3 min 30	Filtrer 26 fois une table coûte plus qu'un seul regroupement : 26 s après correction

Table 11 – Ce que ce projet a appris

14 Feuille de route

1. **Passage au cloud** (projet 6) : lac sur Google Cloud Storage, entrepôt BigQuery, même structure dbt, infrastructure en code.
2. **Modèles dbt incrémentaux** si le volume ou la fréquence augmentent.
3. **Dagster en production** : serveur, démon et base PostgreSQL séparés ; alertes en cas d'échec.
4. **Fraîcheur des sources** : alerte si aucune livraison n'arrive en 35 jours (`dbt source freshness`, déjà configuré).
5. **Analyse des avis** (texte en portugais) pour comprendre les causes d'insatisfaction.

A Arborescence du dépôt

```
src/olist_pipeline/   config, landing, bronze, definitions (Dagster), dbt_cli, report
dbt/models/           staging (9) · intermediate (2) · marts/core (6) · marts/reporting (5)
dbt/tests/            3 rapprochements avec la source
dashboard/app.py      tableau de bord Streamlit (FR/EN)
tests/                pytest + échantillon CI (450 commandes réelles)
dagster_home/         configuration Dagster (concurrence, surveillance)
docs/                 mission, ADR, journal, dictionnaire, rapports, recette
Dockerfile · docker-compose.yml · .github/workflows/ci.yml
```

B Étapes de l'intégration continue

1. Style du code (`ruff`) et tests Python (`pytest`).
2. Livraisons simulées à partir de l'échantillon, puis ingestion dans un lac local (dossier).
3. `dbt build` : 22 modèles, 69 tests de données, 3 tests unitaires.
4. Validation des définitions Dagster, test de fumée du tableau de bord.
5. Construction de l'image Docker.